FrozenLake-v1: Q-Learning Agent

Reinforcement Learning Assignment Report

1. Objective

Implement a tabular reinforcement-learning agent from scratch and train it to navigate the FrozenLake grid from the start tile to the goal while avoiding holes.

2. Environment

Environment: FrozenLake-v1, 4x4 map, is_slippery=False. The deterministic setting makes actions predictable and supports clear evaluation of the learned policy.

3. States, actions and rewards

There are 16 discrete states and 4 actions: left, down, right and up. Reaching the goal gives reward 1; other transitions give reward 0. Episodes end at the goal, a hole or the step limit.

4. State representation

Each grid position is represented by an integer from 0 to 15. The agent stores one Q-value for each state-action pair, producing a 16 x 4 Q-table.

Implementation: Python, Gymnasium, NumPy and Matplotlib.

5. Method and Algorithm

The agent uses tabular Q-Learning, an off-policy temporal-difference method. It explores actions with an epsilon-greedy policy and updates the Q-table after each transition.

Q-learning update rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max Q(s',a') - Q(s,a)]$$

The future-value term is omitted when the episode terminates.

Training hyperparameters

Parameter	Value
Training episodes	20,000
Maximum steps per episode	100
Learning rate (alpha)	0.10
Discount factor (gamma)	0.99
Initial epsilon	1.00
Minimum epsilon	0.02
Epsilon decay	0.9995 per episode
Random seed	42

Exploration gradually decreases so the agent shifts from trying actions to using learned Q-values.

6. Results and Hyperparameter Comparison

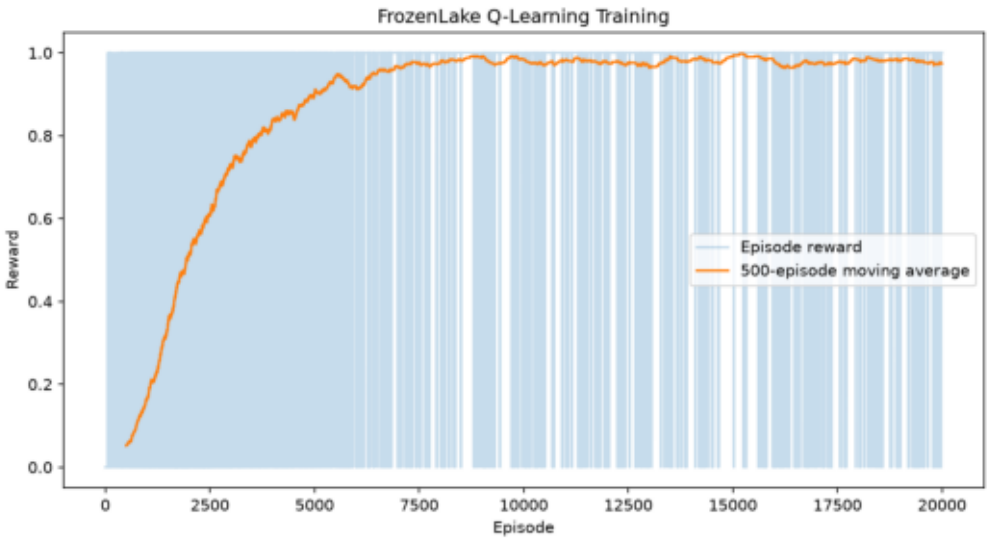
Evaluation episodes: 1000

Successful episodes: 1000

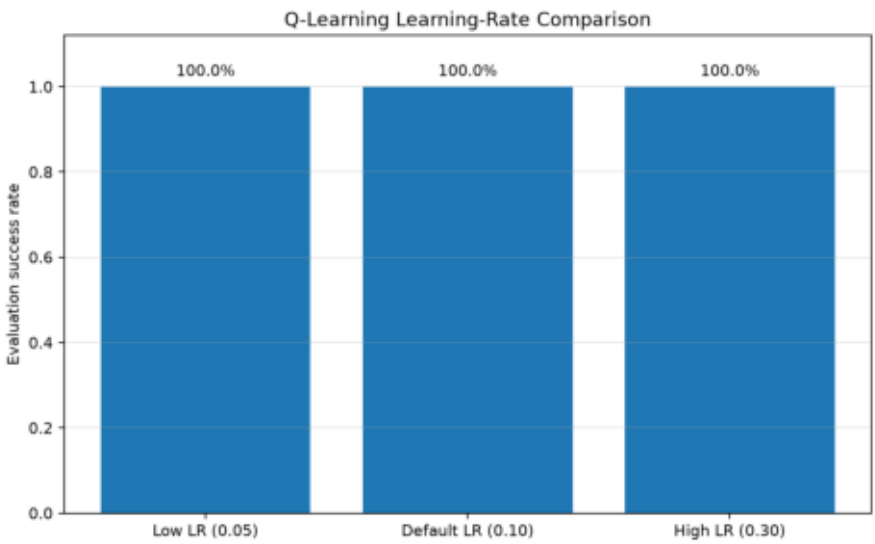
Success rate: 100.0%

Average steps per episode: 6.00

Training reward curve



Learning-rate comparison



7. Discussion, Failure Modes and Conclusion

Observed result

The trained agent achieved a 100.0% success rate across 1000 evaluation episodes, with 6.00 average steps.

Failure modes and limitations

An untrained agent may fall into holes because exploration includes random actions. Learning may be slower with an unsuitable learning rate, insufficient episodes or a poor exploration schedule. The tabular method does not scale well to very large or continuous state spaces. Results on this non-slippery map do not establish performance on stochastic FrozenLake.

Hyperparameter comparison

Three learning rates (0.05, 0.10 and 0.30) were trained and tested using the same environment type. Their measured evaluation success rates are shown in the comparison chart; the best setting is the one with the highest measured success rate.

Conclusion

The project implements Q-Learning from scratch, saves the learned Q-table, records a GIF of the greedy policy, and produces training and comparison plots. The evaluation indicates that the agent learned a successful route in the deterministic 4x4 environment.

Reproducibility and artifacts

The project includes source files, requirements, Q-table, training plots, evaluation JSON, hyperparameter results, a GIF, and an executed Jupyter notebook.